

Übung 27.04. - 01.05.2026
Blatt 02

Besprechung der Aufgaben in der Woche vom 04.05.2026.

my program: *works perfectly*

me: *cleans up the code*

also my program:



Aufgabe 1 [Simple Cell Game]

Implementieren Sie das Simple Cell Game aus Vorlesung 2 (ab Folie 32).

- (a) Ergänzen Sie das Programm um die `printMyRule()` Methode, die den Namen der Wechselregel der jeweiligen Zelle auf die Konsole ausgibt.
- (b) Überladen Sie die Methode aus `testCellBehavior`, indem Sie einen weiteren Übergabeparameter in Form eines `int` ergänzen (also `testCellBehavior(boolean initialState, Rule regel, int z)`). Dieser soll angeben, wie viele Wechselaufforderungen es gibt. Siehe dazu auch Folie 89.

Im Folgenden sollen Sie das Programm um weitere Regeln erweitern. Implementieren Sie die Regel als eigene Klasse, die das Interface `Rule` implementiert. Achten Sie darauf, dass auch die Methode `printRuleName()` sinnvoll umgesetzt wird.

- (c) Ergänzen Sie das Programm um die Parity Rule. Diese Regel setzt den Status auf `true` genau dann, wenn `input` gerade ist. Ansonsten wird der Wert auf `false` gesetzt.
- (d) Ergänzen Sie das Programm um die Conway Rule. Diese Regel besagt, dass wenn der aktuelle Status einer Zelle `true` ist, er auch weiterhin `true` bleibt genau dann, wenn `input` den Wert 2 oder 3 hat. Ansonsten wird der Wert auf `false` gesetzt. Sollte der aktuelle Status einer Zelle zu Beginn `false` sein, wird dieser auf `true` geändert genau dann, wenn `input` den Wert 3 hat.
- (e) Ergänzen Sie das Programm um die Random Rule. Diese Regel bestimmt den nächsten Zustand einer Zelle unabhängig von ihrem aktuellen Zustand. Konkret soll bei jedem Aufruf der Methode `computeNextState` mit gleicher Wahrscheinlichkeit entschieden werden, ob die Zelle im nächsten Schritt den Zustand `true` oder `false` hat. Der Übergabeparameter `input` wird dabei nicht berücksichtigt.
Hinweis: Zur Erzeugung von Zufallswerten können Sie die Methode `Math.random()` verwenden.
- (f) Ergänzen Sie das Programm um eine weitere Regel, die Sie sich selbst ausdenken.

Aufgabe 2 [Notifiable]

In einem Smart-Home-System können Geräte Benachrichtigungen senden. Standardmäßig verwenden alle Geräte eine allgemeine Benachrichtigung, können aber auch spezifische Nachrichten liefern.

Dazu ist das folgende Interface gegeben:

```
public interface Notifiable {
    default String getNotification() {
        return "Test notification! System operating normal.";
    }
}
```

- (a) Implementieren Sie eine Klasse **AlarmSystem**, die das Interface **Notifiable** implementiert. **AlarmSystem** besitzt außerdem den **boolean emergencyMode**, der das System scharf schaltet.
- (b) Überschreiben Sie die Methode **getNotification()** so, dass eine spezifische Nachricht zurückgegeben wird, z. B.: **"Alarm! Please check your home!"**
- (c) Ergänzen Sie die Klasse **AlarmSystem** um die Methode **public String alert()**, die in Abhängigkeit von **emergencyMode** entweder die eigene überschriebene **getNotification()** Methode aufruft oder die Default-Methode aus dem Interface.
- (d) Testen Sie Ihr Programm, indem Sie eine main-Methode schreiben, in der Sie ein Objekt vom Typ **AlarmSystem** erzeugen, die Methode **alert()** aufrufen und das Ergebnis ausgeben. Ändern Sie den Status von **emergencyMode**.

Aufgabe 3 [Annotation]

Ein funktionales Interface ist ein Interface, das genau eine abstrakte Methode besitzt. Es kann zusätzlich weitere Methoden enthalten (z. B. default-Methoden), aber nur eine einzige abstrakte Methode.

Solche Interfaces sind später besonders wichtig, z. B. für Lambda-Ausdrücke – in dieser Aufgabe konzentrieren wir uns jedoch nur auf die Grundlagen.

Gegeben sei das folgende funktionale Interface:

```
@FunctionalInterface
public interface Operation {

    int calculate(int a, int b);

    default String getDescription() {
        return "Default Operation";
    }
}
```

- (a) Erstellen Sie die beiden Klassen **Add** und **Multiply**, welche das Interface **Operation** implementieren und dabei die beiden Methoden passend überschreiben.
- (b) Testen Sie Ihr Programm, indem Sie eine main-Methode schreiben, in der Sie beide Klassen verwenden, Beispielrechnungen durchführen und zusätzlich die Default-Methode **getDescription()** aufrufen.
- (c) Fügen Sie dem Interface eine zweite abstrakte Methode **int calculateTwice(int a, int b);** hinzu. Versuchen Sie zu kompilieren, während die **@FunctionalInterface**-Annotation gesetzt ist. Beobachten Sie den Compilerfehler. Entfernen Sie danach die Annotation und kompilieren Sie erneut. Beantworten Sie danach folgende Fragen:

- Was passiert, wenn ein Interface mit **@FunctionalInterface** mehr als eine abstrakte Methode hat?
 - Was passiert ohne die Annotation?
 - Warum ist die Annotation trotzdem sinnvoll?
- (d) Erstellen Sie eine Klasse **LegacyCalculator** mit einer veralteten Methode **public int add(int a, int b)** und fügen Sie die **@Deprecated**-Annotation hinzu. Verwenden Sie diese Methode in Ihrer main-Methode, sowie in einer weiteren Methode innerhalb Ihrer Main-Klasse. Was beobachten Sie?
- (e) Unterdrücken Sie die Warnung nur für die main-Methode, in der **add(a, b)** aufgerufen wird: Fügen Sie **@SuppressWarnings("deprecation")** unmittelbar vor dem Beginn der main-Methode hinzu. Was beobachten Sie?
- (f) Verschieben Sie die Annotation von der Methode auf die gesamte Klasse. Was beobachten Sie?

Aufgabe 4 [UML]

Erstellen Sie das UML-Diagramm zu dem folgenden Java-Code:

```
public interface Enrollable {
    void enroll(Student student);
}

public class Student {
    private String name;
    private int studentId;

    public Student(String name, int studentId) {
        this.name = name;
        this.studentId = studentId;
    }

    public String getName() {
        return name;
    }
}

public class Course implements Enrollable {
    private String title;
    private Student participant;

    public Course(String title, Student participant) {
        this.title = title;
        this.participant = participant;
    }

    public String getTitle() {
        return title;
    }
}
```

```
@Override
public void enroll(Student student) {
    this.participant = student;
}
}
```